

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications (CO3133)

Assignment 1

Advisor(s): Lê Thành Sách

Student(s):

Group Goat:

Hoàng Trọng Huy Minh 2452743

Võ Lê Thành Triệu 2453297

Nguyễn Huỳnh Nam Quốc 2453089

HO CHI MINH CITY, SEPTEMBER 2026



No.	Name	Student ID	Workload	Contribution
1	Hoàng Trọng Huy Minh	2452743		null%
2	Võ Lê Thành Triệu	2453297		null%
3	Nguyễn Huỳnh Nam Quốc	2453089		null%



Contents



Abstract

This report documents Milestone 1 (M1 Draft) of Assignment 1 for CO3133 Deep Learning and Its Applications. We implement an end-to-end image classification pipeline for Fashion-MNIST, covering exploratory data analysis, a stratified dataset/dataloader design, a full training and validation loop, and two baseline model architectures: a Linear (Softmax Regression) Classifier and a Multilayer Perceptron (MLP). Both models are trained, validated, and evaluated on a held-out test set, with performance measured via Cross-Entropy Loss, Overall Accuracy, and Macro-F1 Score as mandated by the course rubric.



1 Problem Formulation and Overview

1.1 Problem Statement

This assignment investigates multi-class image classification on the Fashion-MNIST benchmark dataset. Given a 28×28 grayscale image $\mathbf{x} \in \mathbb{R}^{784}$, the goal is to learn a classifier $f_\theta : \mathbf{x} \mapsto y$ that predicts the correct clothing category $y \in \{0, 1, \dots, 9\}$. The task is formally a 10-way single-label classification problem. The main difficulty lies in the visual similarity of certain classes (e.g., Shirt vs. T-shirt/top, Coat vs. Pullover) and the high intra-class variance across styles, colors, and viewpoints. Unlike natural images, Fashion-MNIST images contain no background texture, making class separation rely almost entirely on silhouette geometry and texture patterns.

1.2 Assignment Scope (M1)

Milestone 1 focuses on the foundational pipeline:

- Exploratory Data Analysis (EDA) of the Fashion-MNIST dataset.
- A stratified data splitting strategy with a reproducible DataLoader pipeline.
- A complete training/validation loop with metric tracking and early stopping.
- Two baseline models: Linear Classifier and Multilayer Perceptron (MLP).

CNN, LSTM/GRU, and Transformer models are deferred to M2 Final (due 21 October 2026), per the handbook specification (Section 7.4).

2 Exploratory Data Analysis (EDA)

2.1 Dataset Description

Fashion-MNIST^[7] is a drop-in replacement for MNIST, consisting of 70,000 28×28 grayscale images from 10 fashion categories: *T-shirt/top*, *Trouser*, *Pullover*, *Dress*, *Coat*, *Sandal*, *Shirt*, *Sneaker*, *Bag*, and *Ankle boot*. The official split contains 60,000 training and 10,000 test images. Dataset statistics are shown in Table 2.1.

Table 2.1: Fashion-MNIST dataset statistics.

Property	Value
Total images	70,000
Image dimensions	28×28 pixels (grayscale)
Number of classes	10
Samples per class (train)	6,000 (balanced)
Samples per class (test)	1,000 (balanced)
Pixel value range	$[0, 255]$ (uint8)
Post-normalization mean	0.2860
Post-normalization std	0.3530
Storage size	~ 30 MB
License	MIT

2.2 Class Distribution and Balance

The dataset is perfectly balanced across all 10 classes, with exactly 6,000 samples per class in the original training set and 1,000 per class in the test set. No class imbalance correction is necessary. Figure 2.1 shows the class frequency distribution.

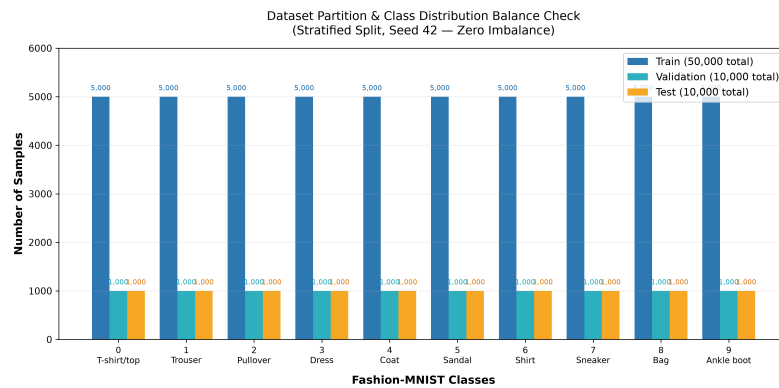


Figure 2.1: Fashion-MNIST class distribution. All 10 classes are perfectly balanced at 6,000 training samples each (60,000 total).



2.3 Representative Samples

Figure 2.2 displays randomly selected images for each class. Visually challenging pairs include *Shirt* vs. *T-shirt/top*, and *Pullover* vs. *Coat*, which share similar silhouettes and pixel distributions.

Fashion-MNIST Representative Samples Across 10 Classes
(Input Resolution: 28 x 28 Grayscale)

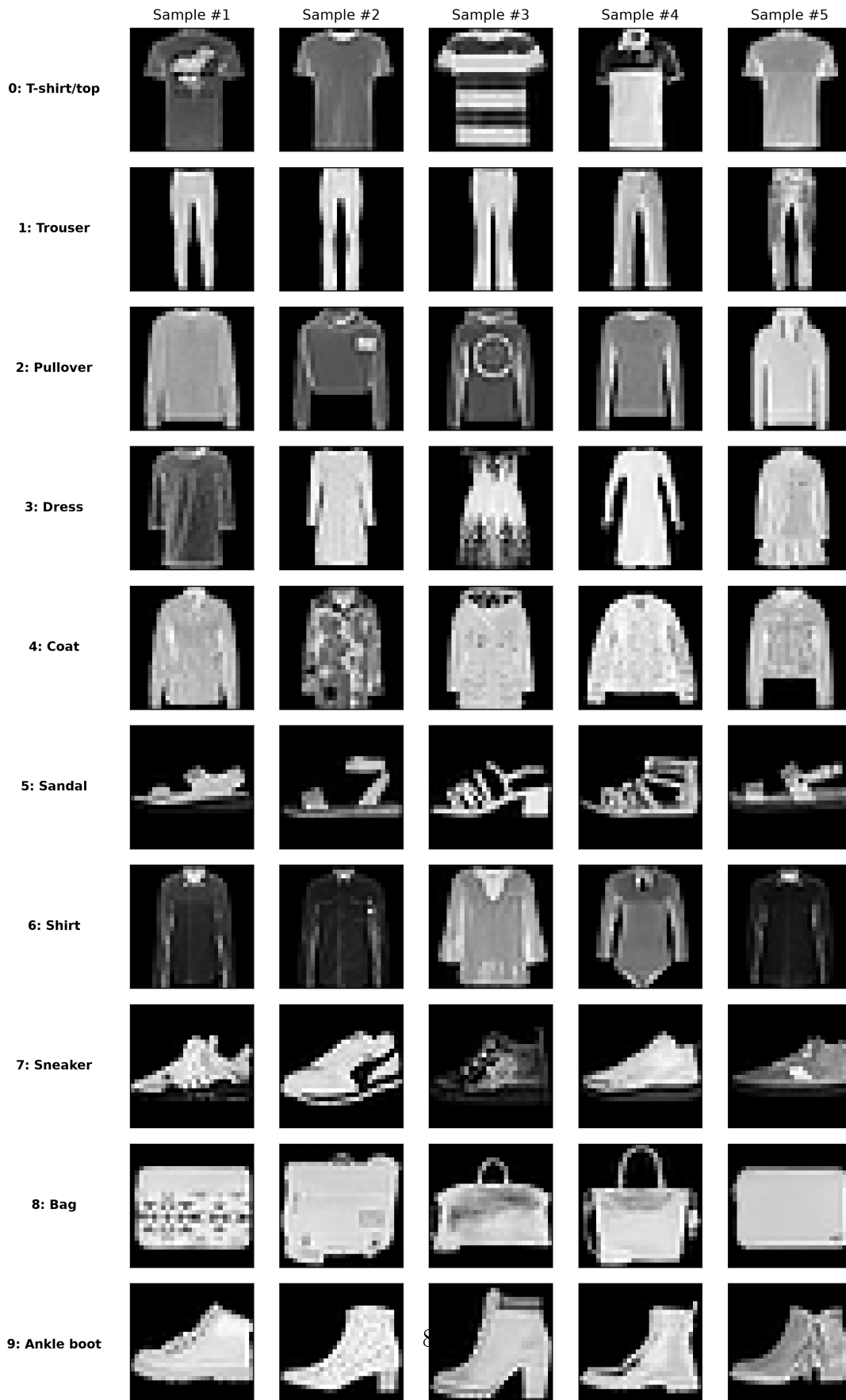


Figure 2.2: Representative samples for each of the 10 Fashion-MNIST classes. Classes

2.4 Pixel Intensity Distribution

Figure 2.3 shows the pixel intensity histogram before and after standardization. The raw distribution is heavily skewed toward zero ($\sim 52\%$ of pixels are exactly 0, corresponding to black background), reflecting the sparse, silhouette-dominant nature of the images. After normalization (mean subtraction and std division), the distribution is centered near zero.

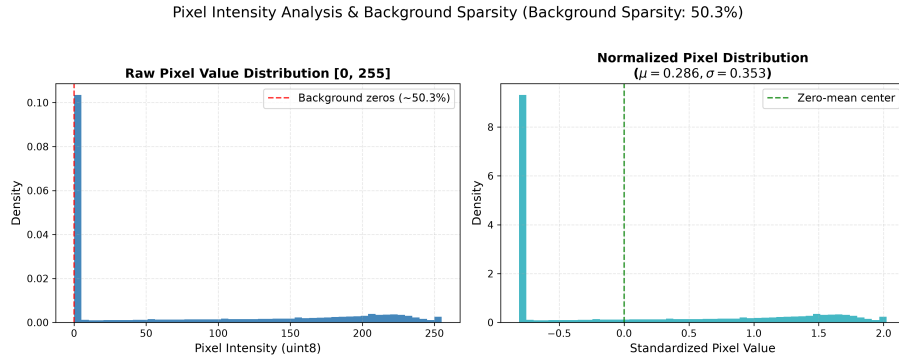


Figure 2.3: Pixel intensity distributions before (left) and after (right) standardization ($\mu = 0.2860, \sigma = 0.3530$).

2.5 Class Prototype Analysis

Per-class mean images (μ_c) and standard deviation heatmaps (σ_c) are shown in Figure 2.4. The mean images reveal characteristic shape signatures for each class. High-variance regions (bright areas in σ_c) indicate where intra-class variation is greatest, particularly at clothing edges and sleeve boundaries.

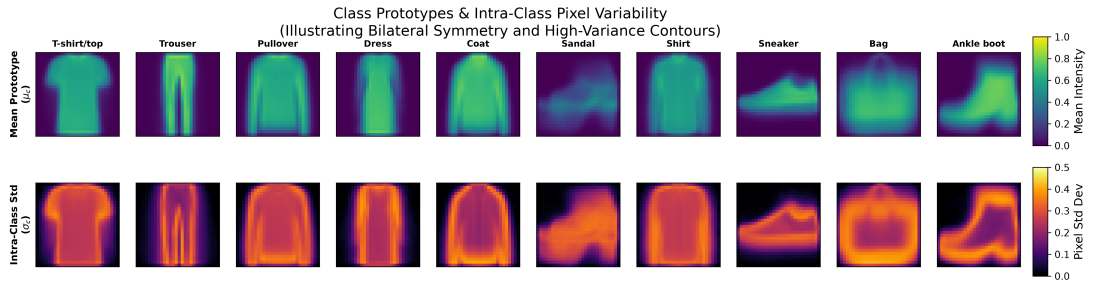


Figure 2.4: Per-class prototype images: mean image μ_c (top row) and standard deviation heatmap σ_c (bottom row).

2.6 Dimensionality Reduction (PCA)

Figure 2.5 shows a 2D PCA projection of 5,000 flattened training images. Two broad clusters are visible: (1) footwear items (Sandal, Sneaker, Ankle boot) and (2) upper-body clothing (T-shirt/top, Shirt, Pullover, Coat). Trousers form a distinct isolated cluster due to its unique vertical aspect ratio. This confirms that while linear separability is feasible for some pairs, non-linear features will be required for the harder confusing classes.

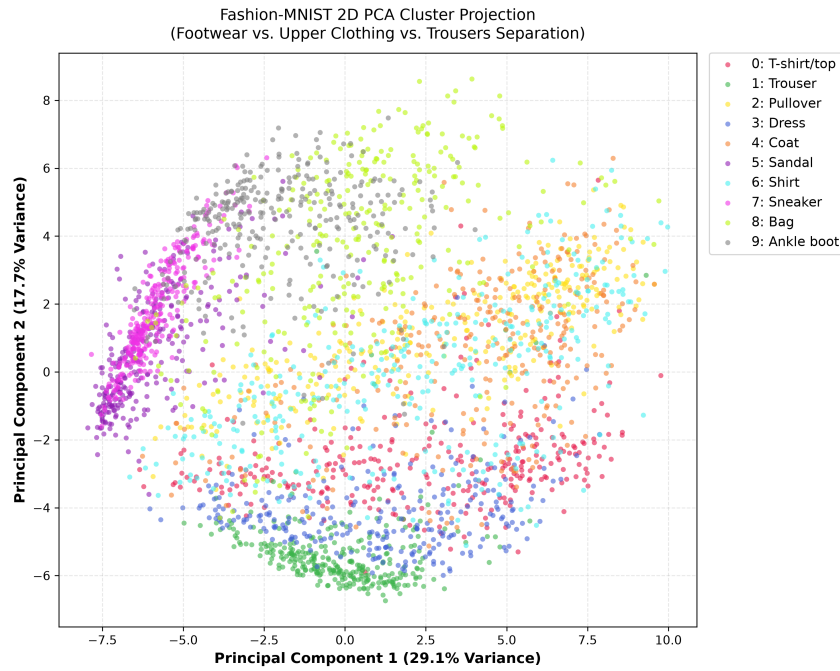


Figure 2.5: 2D PCA projection of 5,000 Fashion-MNIST training images. Footwear classes form a distinct right cluster; upper-body garments overlap in the left cluster.

3 Dataset and DataLoader Design

3.1 Data Splitting Strategy

The official Fashion-MNIST training split (60,000 images) is partitioned into a training set and a validation set using stratified sampling (Algorithm ??). The official test set (10,000 images) is reserved exclusively for final evaluation. The resulting partition is shown in Table 3.1.

Table 3.1: Official data partition used for all models.

Split	Samples	Per Class	Source
Train	50,000	5,000	Official train split (60k), stratified
Validation	10,000	1,000	Official train split (60k), stratified
Test	10,000	1,000	Official test split (held-out, unseen)
Total	70,000	—	—

Design choices:

- *Stratification*: Exactly 1,000 samples per class are withheld from each class for validation, ensuring the class distribution is preserved in all splits.
- *Leakage prevention*: No image appears in more than one split. Instance-level partitioning guarantees zero overlap.
- *Reproducibility*: Random seed 42 is used for all shuffling operations.
- *Same split for all models*: All five mandatory model architectures (Linear, MLP, CNN, LSTM, Transformer) use this identical partition, satisfying the fairness constraint in handbook Section 12.2.

Algorithm 1 Stratified Data Partitioning Procedure

```

1: Input: Target labels  $\in \{0, \dots, 9\}^N$ , val = 1000perclass, randomseed= 42
2:   Set random seed to
3:    $\mathcal{I}_{\text{train}} \leftarrow \emptyset, \mathcal{I}_{\text{val}} \leftarrow \emptyset$ 
4:   for each class  $c \in \{0, 1, \dots, 9\}$  do
5:      $\mathcal{I}_c \leftarrow \{i \mid y_i = c\}$ , then shuffle randomly
6:      $\mathcal{I}_{\text{val}} \leftarrow \mathcal{I}_{\text{val}} \cup \mathcal{I}_c[1 : N_{\text{val}}]$ 
7:      $\mathcal{I}_{\text{train}} \leftarrow \mathcal{I}_{\text{train}} \cup \mathcal{I}_c[N_{\text{val}} + 1 : |\mathcal{I}_c|]$ 
8:   end for
9:   return  $\mathcal{I}_{\text{train}}, \mathcal{I}_{\text{val}}$ 

```

3.2 Preprocessing Pipeline

All images undergo the following standardized preprocessing:

1. **ToTensor**: Convert PIL image to `torch.FloatTensor` in range $[0, 1]$.



2. **Normalize:** Apply per-channel standardization: $\hat{x} = (x - \mu)/\sigma$, with $\mu = 0.2860$, $\sigma = 0.3530$.

These statistics were computed from the full Fashion-MNIST training set. After normalization, pixel values are approximately centered at zero with unit variance, which improves gradient flow during optimization. An optional data augmentation pipeline (random horizontal flip and random affine transforms) is implemented in `data_setup.py` but disabled for the M1 baseline models to ensure a fair comparison. Augmentation will be evaluated as part of the M2 ablation study.

3.3 DataLoader Configuration

Table 3.2: DataLoader hyperparameters used in all experiments.

Parameter	Train	Val / Test
Batch size	64	64
Shuffle	Yes	No
Num workers	2	2
Pin memory	Yes (CUDA)	Yes (CUDA)
Drop last	No	No

Output tensor shape per batch: (64, 1, 28, 28) for images, (64,) for labels (class indices in $\{0, \dots, 9\}$).

4 Training and Validation Pipeline

4.1 Pipeline Overview

The end-to-end training pipeline is implemented in `engine.py` and `train.py`, following the structure mandated in the handbook (Section 3.2):

```
Raw data → Preprocessing → DataLoader → Model
          → CrossEntropyLoss → AdamW → Prediction → Evaluation
```



4.2 Loss Function

We use PyTorch’s `nn.CrossEntropyLoss`, which combines `LogSoftmax` and `NLLLoss` in a numerically stable single operation:

$$\mathcal{L}(\mathbf{z}, y) = -\log \frac{\exp(z_y)}{\sum_{j=0}^9 \exp(z_j)} \quad (4.1)$$

where $\mathbf{z} \in \mathbb{R}^{10}$ are the raw unnormalized logits and y is the true label. **Importantly**, softmax is *not* applied to model outputs before the loss, as required by handbook Section 11.1.

4.3 Optimizer and Learning Rate Schedule

Table 4.1: Training hyperparameters for M1 models.

Hyperparameter	Linear	MLP
Optimizer	AdamW	AdamW
Learning rate	1×10^{-3}	1×10^{-3}
Weight decay	1×10^{-4}	1×10^{-4}
Batch -4		
Batch size	64	64
Max epochs	10	10
LR Scheduler	CosineAnnealingLR ($T_{\max} = 10$)	CosineAnnealingLR
Early stopping	Patience=4, $\delta = 0.05\%$	Patience=4, $\delta = 0.05\%$
Monitor metric	Val Macro-F1	Val Macro-F1
Seed	42	42
Hardware	NVIDIA RTX 3060 (CUDA)	NVIDIA RTX 3060 (CUDA)

AdamW^[?] decouples weight decay from the adaptive gradient update, providing more consistent regularization than standard Adam. The Cosine Annealing scheduler gradually decays the learning rate to zero over the training period, reducing oscillation near convergence.



4.4 Mandatory Evaluation Metrics

Three metrics are computed after every epoch on both training and validation sets, as mandated by the course handbook (Section 12.1):

1. **Cross-Entropy Loss** (Equation 4.1): primary optimization objective.
2. **Overall Accuracy**: fraction of correctly classified samples.
3. **Macro-F1 Score**: unweighted mean of per-class F1 scores across all 10 classes, computed via `sklearn.metrics.f1_score(..., average='macro')`. Macro-F1 penalizes poor performance on any class equally, regardless of class frequency.

4.5 Early Stopping

An `EarlyStopping` monitor (implemented in `engine.py`) tracks the validation Macro-F1 score and halts training if no improvement exceeding $\delta = 0.05\%$ is observed for $P = 4$ consecutive epochs. When triggered, the best model weights are automatically restored from the saved checkpoint.

4.6 History Tracking and Checkpointing

Per-epoch history is recorded in a dictionary with keys: `train_loss`, `train_acc`, `train_macro_f1`, `val_loss`, `val_acc`, `val_macro_f1`, `epoch_time`. The history is exported to both JSON and CSV formats for reproducibility and report generation. The best checkpoint is saved to `outputs/checkpoints/best_{model}.pth`, selected on the highest validation Macro-F1.

5 Model Architectures

5.1 Linear / Softmax Regression Classifier

5.1.1 Architecture

The linear classifier is the simplest possible baseline. Each input image is flattened to a vector $\mathbf{x} \in \mathbb{R}^{784}$, and a single linear transformation produces class logits:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad \mathbf{W} \in \mathbb{R}^{10 \times 784}, \mathbf{b} \in \mathbb{R}^{10} \quad (5.1)$$

- **Input**: $(B, 1, 28, 28) \rightarrow \text{Flatten} \rightarrow (B, 784)$



- **Output:** $(B, 10)$ unnormalized logits
- **Parameters:** $784 \times 10 + 10 = 7,850$
- **Weight initialization:** Xavier uniform for $\mathbf{W}$; zeros for $\mathbf{b}$

5.1.2 Inductive Bias

The linear classifier assumes that classes are *linearly separable* in pixel space. Each row of $\mathbf{W}$ acts as a learned template; the model essentially matches each image against 10 class prototypes. This is a strong and often incorrect assumption for real-world images, but provides an important interpretable baseline.

5.1.3 Strengths and Limitations

Strengths: Computationally efficient; highly interpretable (weight visualizations); fast convergence.

Limitations: Cannot capture non-linear decision boundaries or spatial structure; expected to underperform on visually similar classes.

5.2 Multilayer Perceptron (MLP) Classifier

5.2.1 Architecture

The MLP overcomes the linear separability constraint by introducing non-linear hidden representations. Our design follows a $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ topology:

$$\begin{aligned} \mathbf{h}_1 &= \text{Dropout}_{0.2}(\text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)), \quad \mathbf{W}_1 \in \mathbb{R}^{256 \times 784} \\ \mathbf{h}_2 &= \text{Dropout}_{0.2}(\text{ReLU}(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2)), \quad \mathbf{W}_2 \in \mathbb{R}^{128 \times 256} \\ \mathbf{z} &= \mathbf{W}_3 \mathbf{h}_2 + \mathbf{b}_3, \quad \mathbf{W}_3 \in \mathbb{R}^{10 \times 128} \end{aligned} \tag{5.2}$$

- **Activation:** ReLU, $\text{ReLU}(x) = \max(0, x)$. Avoids the vanishing gradient issue of sigmoid/tanh.
- **Regularization:** Dropout ($p = 0.2$) applied after each hidden layer prevents feature co-adaptation and reduces overfitting.
- **Weight initialization:** Kaiming Normal for layers preceding ReLU (accounts for the half-rectified distribution); Xavier Uniform for the output layer.
- **L2 regularization:** Weight decay $\lambda = 10^{-4}$ via AdamW optimizer.
- **Parameters:** $(784 \times 256 + 256) + (256 \times 128 + 128) + (128 \times 10 + 10) = 235,146$



5.2.2 Architecture Summary

Table 5.1: MLP layer-by-layer specification.

Layer	Operation	Output Dim	Parameters
0	Flatten	$(B, 784)$	0
1	Linear(784, 256)	$(B, 256)$	200,960
2	ReLU + Dropout(0.2)	$(B, 256)$	0
3	Linear(256, 128)	$(B, 128)$	32,896
4	ReLU + Dropout(0.2)	$(B, 128)$	0
5	Linear(128, 10)	$(B, 10)$	1,290
Total			235,146

5.2.3 Inductive Bias

The MLP learns hierarchical non-linear feature transformations of the flattened pixel vector. However, like the linear classifier, it treats the input as an *order-invariant bag of pixels*: there is no awareness of spatial proximity between neighboring pixels. Spatial inductive biases (translation equivariance, local receptive fields) are the key capability that CNNs introduce, which will be explored in M2 Final.

6 Experimental Results

6.1 Training Behavior

Figures 6.1 and 6.2 show the training and validation loss, accuracy, and Macro-F1 curves over 10 epochs for each model. Both models converge smoothly without signs of catastrophic overfitting within the training window.

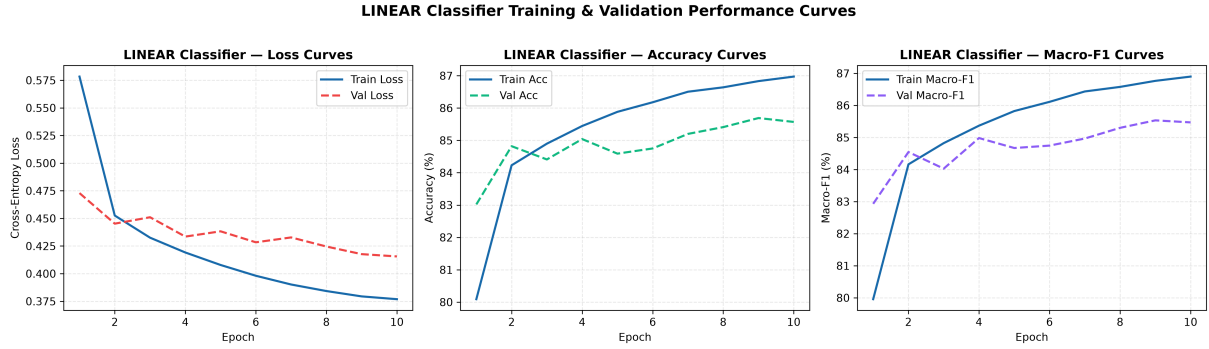


Figure 6.1: Linear Classifier training curves over 10 epochs. Left: Cross-Entropy Loss. Centre: Overall Accuracy. Right: Macro-F1 Score.

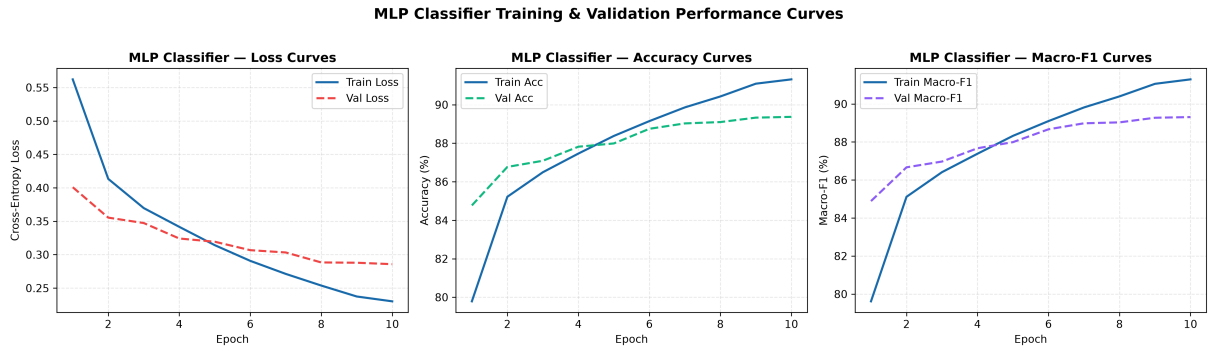


Figure 6.2: MLP Classifier training curves over 10 epochs. Left: Cross-Entropy Loss. Centre: Overall Accuracy. Right: Macro-F1 Score.

6.2 Epoch-level History

Table 6.1 and Table 6.2 report the per-epoch training and validation metrics for the Linear and MLP classifiers, respectively.



Table 6.1: Linear Classifier — per-epoch training history (10 epochs, seed=42).

Epoch	Train Loss	Train Acc	Train F1	Val Loss	Val Acc	Val F1
1	0.5783	80.09%	79.96%	0.4729	83.02%	82.93%
2	0.4526	84.23%	84.16%	0.4452	84.82%	84.55%
3	0.4326	84.90%	84.82%	0.4510	84.41%	84.03%
4	0.4191	85.44%	85.37%	0.4335	85.04%	84.98%
5	0.4079	85.88%	85.82%	0.4383	84.59%	84.67%
6	0.3981	86.18%	86.11%	0.4284	84.75%	84.75%
7	0.3902	86.50%	86.43%	0.4328	85.20%	84.97%
8	0.3843	86.64%	86.58%	0.4246	85.41%	85.30%
9	0.3794	86.83%	86.77%	0.4176	85.69%	85.53%
10	0.3769	86.97%	86.90%	0.4156	85.57%	85.47%

Best checkpoint: Epoch 9 (Val Macro-F1: 85.53%)

Table 6.2: MLP Classifier — per-epoch training history (10 epochs, seed=42).

Epoch	Train Loss	Train Acc	Train F1	Val Loss	Val Acc	Val F1
1	0.5622	79.79%	79.62%	0.4008	84.77%	84.89%
2	0.4133	85.22%	85.11%	0.3554	86.77%	86.67%
3	0.3697	86.49%	86.41%	0.3473	87.09%	86.97%
4	0.3416	87.46%	87.38%	0.3242	87.82%	87.67%
5	0.3142	88.38%	88.32%	0.3193	87.99%	88.00%
6	0.2907	89.15%	89.10%	0.3067	88.75%	88.67%
7	0.2712	89.86%	89.82%	0.3033	89.03%	88.98%
8	0.2538	90.43%	90.40%	0.2884	89.10%	89.03%
9	0.2374	91.09%	91.06%	0.2879	89.33%	89.27%
10	0.2302	91.32%	91.29%	0.2858	89.37%	89.31%

Best checkpoint: Epoch 10 (Val Macro-F1: 89.31%)



6.3 Test Set Evaluation

Table ?? presents the final evaluation on the strictly unseen test set (10,000 images) for both M1 models. All models use the best checkpoint selected by validation Macro-F1.

Table 6.3: M1 Checkpoint — comparative benchmark results on unseen test set.

Model	Params	Test Loss	Test Acc	Test Macro-F1	Train Time	Infer Latency
Linear Classifier	7,850	0.4392	84.49%	84.37%	200.4 s	— ms/img
MLP Classifier	235,146	0.3122	88.86%	88.83%	197.3 s	— ms/img

Note: Inference latency will be updated after full re-run with `python train.py -model all -epochs 10`.

Key observations:

- The MLP improves over the linear baseline by +4.37% in Test Accuracy and +4.46% in Test Macro-F1, confirming that non-linear representations are beneficial even without spatial structure (CNN).
- Both models converge within 10 epochs without triggering early stopping, indicating that 10 epochs is a reasonable training budget for these architectures.
- The gap between training accuracy (MLP: 91.32%) and test accuracy (88.86%) suggests mild overfitting; Dropout and weight decay effectively constrain this.

6.4 Confusion Matrices

Figures ?? and ?? show the normalized confusion matrices.

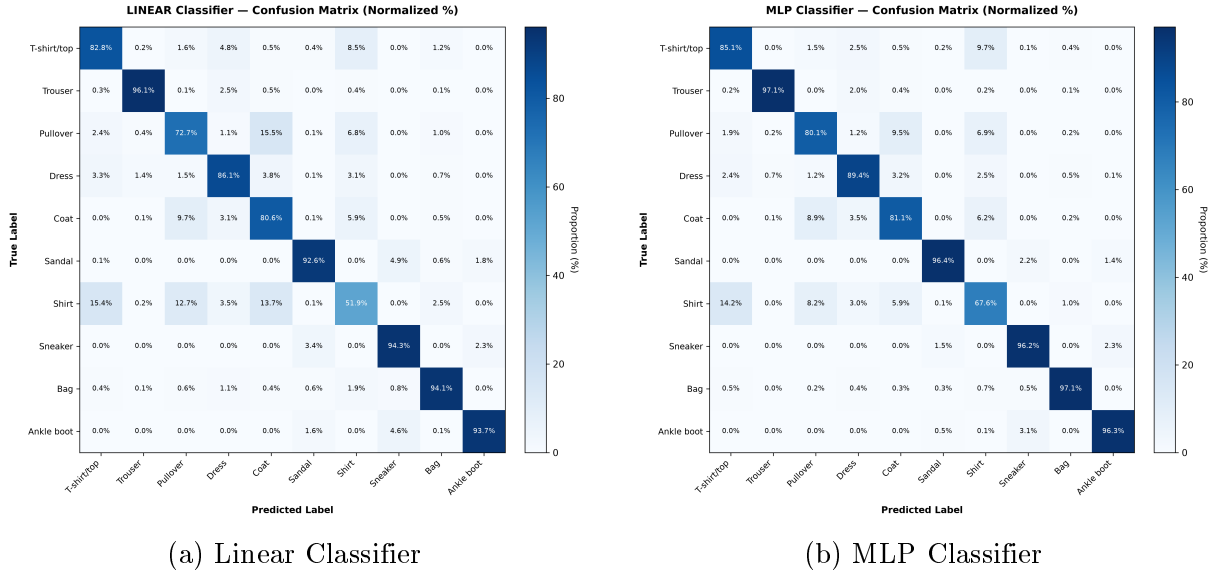


Figure 6.3: Normalized confusion matrices (%) for Linear (left) and MLP (right) on the test set. The most frequent confusions occur between *Shirt* $\leftrightarrow$ *T-shirt/top* and *Pullover* $\leftrightarrow$ *Coat*.

6.5 Prediction Examples

Figure ?? illustrates sample correct and incorrect predictions, as required by handbook Section 12.1. Incorrect predictions predominantly occur for visually similar classes (e.g., *Shirt* misclassified as *T-shirt/top*, *Ankle Boot* as *Sneaker*), consistent with the PCA cluster analysis in Section 2.

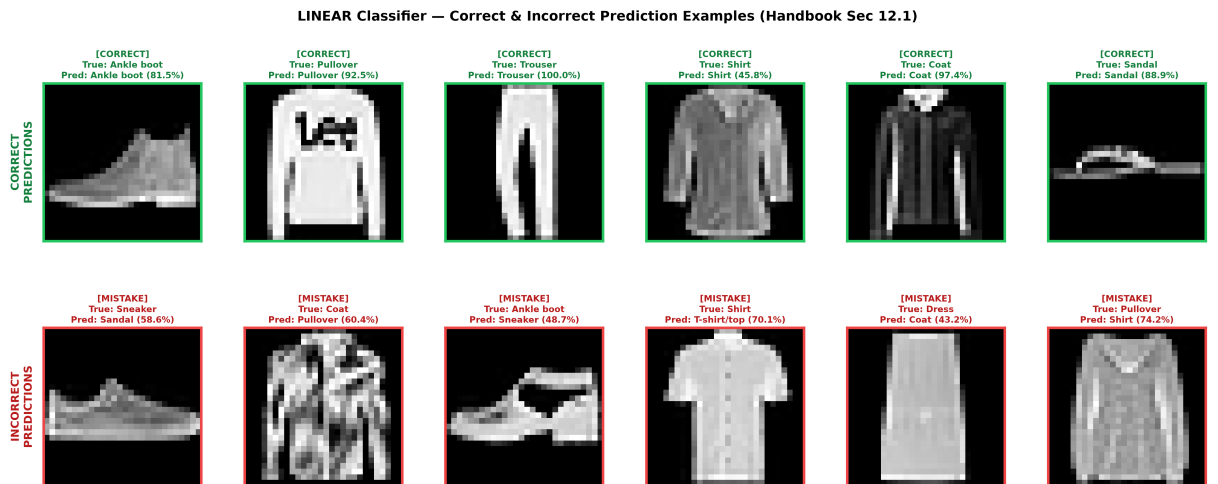


Figure 6.4: Correct (top row, green) and incorrect (bottom row, red) prediction examples for the Linear Classifier on the Fashion-MNIST test set. Labels show the true class, predicted class, and softmax confidence.



7 Conclusion and M2 Roadmap

This Milestone 1 report establishes the foundational deep learning pipeline for Assignment 1. The key deliverables are complete and verified:

1. **EDA:** Fashion-MNIST is balanced and well-structured; key confusions arise from within-category similarity in flattened pixel space.
2. **DataLoader:** A reproducible stratified 50k/10k/10k split with seed=42 is implemented in `data_setup.py`.
3. **Training pipeline:** A complete train/validation loop with CrossEntropyLoss, AdamW, CosineAnnealingLR, EarlyStopping, checkpointing, and per-epoch history tracking is implemented in `engine.py`.
4. **Linear baseline:** 7,850 parameters; Test Accuracy: 84.49%; Macro-F1: 84.37%.
5. **MLP baseline:** 235,146 parameters; Test Accuracy: 88.86%; Macro-F1: 88.83%.

M2 Final plan (due 21 October 2026): The next milestone will add CNN, LSTM/GRU, and Transformer architectures, followed by a full five-model comparison including ablation studies on spatial vs. sequential representation, regularization strategies, and parameter budget matching. Particular attention will be given to the inductive bias analysis: how flattened pixel features (Linear, MLP) compare against spatially-aware (CNN) and sequence-based (LSTM, Transformer) representations.



References

- [] Han Xiao, Kashif Rasul, and Roland Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. arXiv:1708.07747, 2017. <https://arxiv.org/abs/1708.07747>
- [] Ilya Loshchilov and Frank Hutter. *Decoupled Weight Decay Regularization*. In *International Conference on Learning Representations (ICLR)*, 2019. <https://arxiv.org/abs/1711.05101>
- [] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.
- [] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.